

TANGO: Token-Aggregated Nonlinear Gating Operators for Natural and Formal Language Modeling

Joshua Nunley

Department of Informatics

Luddy School of Informatics, Computing, and Engineering

Cognitive Science Program

Indiana University Bloomington

joshnunl@iu.edu

Abstract

A standard Transformer block separates cross-token interaction in self-attention from the nonlinear feed-forward network applied independently at each position. We introduce Token-Aggregated Nonlinear Gating Operators (TANGO), which replaces these two sublayers with a single cross-token gated residual update. Each source token produces a Swish-gated linear unit (SwiGLU) gate vector. Query-key similarities determine a weighted average of the source gates available to each destination, and the resulting gate rescales projected features at that destination before they are added to the residual stream. The TANGO model computes a separate weight for every causally visible source. Windowed Aggregation of Nonlinear Gating Operators (WANGO) retains the same unnormalized pairwise scores within a recent window. For older sources, it uses a positive feature-map weighting rule whose required statistics can be maintained with running sums. TANGO is quadratic in sequence length. For fixed window and feature dimensions, the WANGO model is linear in sequence length.

We compare the proposed models with recurrent and untied Transformer++ baselines, a full-attention Gated Attention Unit (GAU), and FLASH. The TANGO, WANGO, and Recurrent Transformer++ models apply one shared block four times, whereas Untied Transformer++, GAU, and FLASH use four independently parameterized blocks. We train three runs of each architecture. Across architectures, we match the number of parameters outside the tied token embedding/output matrix (approximately 44.3M), the training objective, the example order, and the total amount of training data. On FineWeb-Edu, the WANGO model obtains the lowest mean validation negative log-likelihood (NLL) among models whose computation is linear in sequence length. It also has lower mean NLL than Recurrent Transformer++ at nearly the same analytical forward-pass multiply-accumulate count. The TANGO model obtains the lowest mean validation NLL on FineWeb-Edu, Lean, and DeepMind Mathematics, but has the largest analytical forward-pass operation count among the compared architectures.

1 Introduction

A standard Transformer block contains two sublayers with different roles. Self-attention combines information across token positions, and a feed-forward network then applies the same nonlinear transformation independently at each position [28]. We introduce a block that replaces this self-attention-feed-forward pair with a single cross-token gated residual update. The construction starts from the Swish-gated linear unit (SwiGLU) [24]. A conventional SwiGLU computes both a gate and the features modulated by that gate from the same token representation. Our block instead allows a gate computed at one position to modulate projected features at another. We call the position that supplies a gate the source and the position whose features it modulates the destination. For source representation x_j and destination representation x_i , the corresponding update is

$$f(x_i; x_j) = W_o[\text{SiLU}(W_g x_j) \odot W_v x_i]. \quad (1)$$

The gate is generated nonlinearly from the source representation. Once generated, it acts as a diagonal scaling operator on the destination features $W_v x_i$. For each destination, query-key similarities define nonnegative

weights over the current and earlier source positions. The model averages the corresponding gate vectors and applies the result to the destination features. Source tokens therefore determine feature-wise scaling at a destination rather than supplying the value vectors averaged by self-attention. They do not generate arbitrary projection matrices.

We call the full-prefix architecture Token-Aggregated Nonlinear Gating Operators (TANGO). Its block uses the update above in place of separate self-attention and position-wise feed-forward sublayers. We also introduce Windowed Aggregation of Nonlinear Gating Operators (WANGO), a variant whose computation is linear in sequence length for fixed window and feature dimensions. Section 2 gives the complete definitions.

We consider two methods for computing the weights assigned to source gates. The TANGO model computes softmax weights from scaled cosine similarities between every destination and every causally visible source. The WANGO model retains the same exponential similarity scores for sources in a recent window. For older sources, it computes a different query-key score using a positive feature map. This score factorizes into query and key features, so the model can maintain running sums of transformed keys and their outer products with gate vectors. Every earlier source can still contribute to the update. The two models use the same parameterization and apply the aggregated gate in the same way; they differ only in how they compute weights for older sources. The full-prefix rule is quadratic in sequence length, whereas the windowed rule is linear when the window and feature dimensions are fixed.

We compare the proposed architectures with Recurrent Transformer++, Untied Transformer++, the full-attention Gated Attention Unit (GAU), and FLASH [14]. We use “Transformer++,” as in Gu and Dao [10], to denote a modern decoder block with pre-RMSNorm, bias-free self-attention using rotary positional embeddings, and a SwiGLU feed-forward network. Recurrent Transformer++ applies one such block four times with shared parameters, as do the proposed models with their respective blocks. Untied Transformer++ instead uses four independently parameterized Transformer blocks. GAU attends over every causal token pair, whereas FLASH combines local pairwise attention with a global linear-attention summary. All models have a 512-dimensional residual stream. We adjust each architecture’s internal hidden dimensions to match approximately 44.3 million distinct parameters outside the tied token embedding/output matrix, counting reused weights once. We train each architecture with the same three paired initialization and data-order seed configurations. Architectures with a given data-order seed receive exactly the same training examples in the same order.

On 8,192-token FineWeb-Edu language modeling, the TANGO model obtains the lowest mean validation negative log-likelihood (NLL) overall. The WANGO model has the lowest mean NLL among architectures with computation linear in sequence length. It also has lower mean NLL than Recurrent Transformer++, whose analytical forward-pass multiply-accumulate count is nearly identical at this context length. The TANGO model also obtains the lowest mean validation NLL on both components of the Lean evaluation—causal modeling of source files from Lean’s Mathlib theorem library and proof completion—and on DeepMind Mathematics, evaluated across all 56 modules [21].

Our contributions are:

- We introduce the TANGO model, which replaces self-attention and the position-wise feed-forward network with one gated update. Source tokens produce SwiGLU gates, and each destination applies their content-weighted average to its own projected features.
- We develop the WANGO model, which uses direct pairwise weighting within a recent window and running sums derived from a positive feature map for older sources. Its computation is linear in sequence length for fixed dimensions.
- We compare six architectures under matched nonembedding parameter counts, training objectives, data order, and training budgets. Among architectures with computation linear in sequence length, the windowed model has the lowest mean FineWeb-Edu validation NLL. The full-prefix model has the lowest mean validation NLL overall on all three benchmarks.

2 Model

2.1 Source-conditioned SwiGLU gating

In each repeated block, TANGO and WANGO replace the standard causal self-attention sublayer and position-wise feed-forward sublayer with one source-conditioned gated update. We define the update below.

Let $h_i \in \mathbb{R}^d$ be the residual representation at position i , and let $x_i = \text{RMSNorm}(h_i)$, where RMSNorm denotes root-mean-square normalization. A conventional SwiGLU constructs a gate vector and a projected feature vector from the same normalized representation. Our update allows the gate and the projected features to come from different positions. Each position produces a SwiGLU gate vector and a projected feature vector,

$$g_i = \text{SiLU}(W_g x_i) \in \mathbb{R}^F, \quad v_i = W_v x_i \in \mathbb{R}^F, \quad (2)$$

where F is the gated hidden width. For each destination i , we use content-dependent weights to average the gate vectors from causally visible source positions; we denote this average by $\bar{g}_i$. The model then multiplies $\bar{g}_i$ elementwise by the destination’s projected feature vector v_i :

$$h_i^+ = h_i + W_o(\bar{g}_i \odot v_i). \quad (3)$$

If all weight is assigned to source j , the residual update is $W_o \text{diag}(g_j) W_v x_i$. The components of g_j scale the corresponding components of $W_v x_i$ before the shared output projection W_o is applied.

We perform gate aggregation independently in H heads. We split each gate into $g_{ia} \in \mathbb{R}^{F/H}$ and form a query q_{ia} and key $k_{ia} \in \mathbb{R}^{d_h}$ from x_i , where $d_h = d/H$. We apply rotary positional embeddings (RoPE) [26] and then normalize each query and key to unit length. For each causally visible pair $j \leq i$, head a assigns the logit

$$\ell_{ija} = \tau_a q_{ia}^\top k_{ja}, \quad \tau_a = \exp(\min(\theta_a, \log 20)), \quad (4)$$

with one learned temperature τ_a per head.

Each head also has a learned no-update option, represented by a gate vector of zero. We refer to this as the null gate. Assigning weight to it reduces the magnitude of the residual update. Let $\mathcal{C}_i$ be the set of source positions visible from destination i , and let $n_i = |\mathcal{C}_i|$. The null gate has unnormalized weight $n_i \exp(b_a)$ for head a , where b_a is learned. Scaling this weight by the number of visible sources means that adding more source positions does not by itself force the normalized weight on the null gate toward zero. For nonnegative unnormalized source weights m_{ija} , the aggregated gate is

$$\bar{g}_{ia} = \frac{\sum_{j \in \mathcal{C}_i} m_{ija} g_{ja}}{\sum_{j \in \mathcal{C}_i} m_{ija} + n_i \exp(b_a)}, \quad \bar{g}_i = \text{concat}_{a=1}^H \bar{g}_{ia}. \quad (5)$$

Because the null gate is zero, its weight appears in the denominator but adds nothing to the numerator. Any position can contribute its gate to itself and to later positions. We use “source” and “destination” only to distinguish the two positions in a causal pair; they are not different kinds of tokens.

The gate can also be viewed as an operator on the projected destination features. For head a , let $D_{ja} = \text{diag}(g_{ja})$. If α_{ija} denotes the normalized coefficient of source j in equation (5), then $\text{diag}(\bar{g}_{ia}) = \sum_{j \in \mathcal{C}_i} \alpha_{ija} D_{ja}$. Thus, each destination forms a content-dependent, headwise average of diagonal feature-scaling operators induced by the visible source positions. A fixed D_{ja} acts linearly on the corresponding projected destination features. “Nonlinear” in the name Token-Aggregated Nonlinear Gating Operators refers to the SiLU computation that produces each source gate and to the content-dependent aggregation weights, not to the action of a fixed diagonal operator. Source positions therefore control diagonal feature scalings between the shared projections W_v and W_o ; they do not generate arbitrary projection matrices.

2.2 Full-prefix and windowed gate aggregation

TANGO: full-prefix aggregation. The TANGO model assigns

$$m_{ija}^{\text{full}} = \exp(\ell_{ija}) \quad (6)$$

to every visible source. Substituting these unnormalized weights into equation (5) is equivalent to a softmax over the source logits ℓ_{ija} and a null-option logit $b_a + \log n_i$. Because queries and keys have unit norm,

the source logits are temperature-scaled cosine similarities. Standard self-attention uses analogous weights to average value vectors. The TANGO model instead uses them to average the F -dimensional SwiGLU gates, which then modulate the destination’s projected feature vector. Computing all query–key scores costs $\Theta(T^2d)$, and aggregating gates over all source–destination pairs costs $\Theta(T^2F)$ for a sequence of length T . The full-prefix model is therefore quadratic in sequence length.

WANGO: windowed aggregation. The WANGO model partitions each causal prefix into a trailing window $\mathcal{L}_i$ and an older prefix $\mathcal{G}_i$:

$$\mathcal{L}_i = \{j : \max(0, i - W + 1) \leq j \leq i\}, \quad \mathcal{G}_i = \{j : 0 \leq j < i - W + 1\}. \quad (7)$$

For sources in $\mathcal{L}_i$, the model uses the same unnormalized exponential weights as equation (6). For older sources, it uses the deterministic positive feature map

$$\phi(u) = \frac{\max(\text{ELU}(u) + 1 + \epsilon, \epsilon)}{\sqrt{d_h}}, \quad \epsilon = 10^{-4}, \quad (8)$$

where ELU and max are applied elementwise, and assigns

$$m_{ija}^{\text{old}} = \phi(\sqrt{\tau_a} q_{ia})^\top \phi(\sqrt{\tau_a} k_{ja}). \quad (9)$$

The gate contributions from recent and older sources enter one numerator, and their weights and the null-gate weight enter one denominator. We therefore normalize all contributions together rather than normalizing the recent and older contributions separately. Recent pairs retain the TANGO model’s unnormalized exponential cosine-similarity scores, and every older source continues to contribute through the factorized score in equation (9).

The feature-map score factorizes into query and key features, so the contribution from older tokens can be computed from running prefix sums. For each head, define

$$S_{ia} = \sum_{j \in \mathcal{G}_i} \phi(\sqrt{\tau_a} k_{ja}) g_{ja}^\top, \quad z_{ia} = \sum_{j \in \mathcal{G}_i} \phi(\sqrt{\tau_a} k_{ja}). \quad (10)$$

Each term in S_{ia} is an outer product between a transformed key and a source gate subvector. Thus, S_{ia} is the running sum of these outer products, while z_{ia} is the running sum of the transformed keys. The older-prefix numerator and denominator are then $\phi(\sqrt{\tau_a} q_{ia})^\top S_{ia}$ and $\phi(\sqrt{\tau_a} q_{ia})^\top z_{ia}$. When moving from position i to $i + 1$, the token that leaves the trailing window is added to these sums. Our experiments use $W = 64$ and compute the prefix sums with a blockwise parallel scan using blocks of 64 positions. For fixed W , d , and F , both the local pairwise computation and the older-prefix computation are linear in T . The two architectures use the same parameterization, have the same parameter count, and apply the aggregated gate in the same way. They are trained separately and do not share learned parameter values. Replacing full-prefix aggregation with the windowed rule adds no trainable parameters.

2.3 Block sharing and comparison architectures

All models use $d = 512$. The proposed models use $H = 16$ heads with $d_h = 32$. Within each proposed model, one parameterized block is applied four times,

$$\mathbf{h}^{(r+1)} = \text{Block}_\theta(\mathbf{h}^{(r)}), \quad r \in \{0, 1, 2, 3\}. \quad (11)$$

The block receives no embedding that identifies the application number. Stochastic depth is not used, and the original token embeddings are not re-added between repetitions. A final RMSNorm and a tied input-embedding/output matrix produce the language-model logits.

We compare against four architectures:

- **Recurrent Transformer++.** One pre-RMSNorm decoder block is reused four times in sequence. Each pass applies causal multi-head self-attention with RoPE followed by SwiGLU applied independently at each position. It matches the proposed models’ parameter sharing and number of sequential block applications.

Table 1: Parameter matching across architectures. Each model performs four sequential block applications and has approximately 44.3M nonembedding parameters. F is the intermediate width used by the proposed models and Transformers; E is the GAU or FLASH expansion width. Reused parameters are counted once. GAU is listed separately for each dataset because its learned relative-position parameters depend on context length; the other configurations are identical across datasets.

Architecture	Block sharing	F or E	Nonembedding parameters
TANGO	1 block reused 4 times	28,480	44,270,624
WANGO	1 block reused 4 times	28,480	44,270,624
Recurrent Transformer++	1 block reused 4 times	28,144	44,279,296
Untied Transformer++	4 independent blocks	6,528	44,306,944
Full-attention GAU (FineWeb-Edu)	4 independent blocks	7,136	44,237,564
Full-attention GAU (Lean)	4 independent blocks	7,152	44,286,844
Full-attention GAU (Mathematics)	4 independent blocks	7,152	44,272,508
FLASH	4 independent blocks	7,152	44,274,556

- **Untied Transformer++.** Four independently parameterized blocks use the same Transformer equations, with no parameter sharing across layers.
- **Full-attention GAU.** Four independent Gated Attention Unit (GAU) blocks use a 128-dimensional query/key projection and ReLU-squared attention over every causal token pair.
- **FLASH.** Four independent blocks combine full pairwise GAU attention within 256-token chunks with a global linear-attention summary, following Hua et al. [14]. Its computation is linear in sequence length for a fixed chunk size.

Parameter matching. Each architecture targets 44,268,416 distinct nonembedding parameters, counting reused weights once. At hidden width 512, the tied input-embedding/output matrix for the 50,257-entry GPT-2 vocabulary contains 25,731,584 parameters, leaving 44,268,416 parameters in a 70M-parameter model for the nonembedding target. We choose the intermediate or expansion width separately for each architecture; every resulting parameter count is within 0.1% of the target.

The approximately 70M count applies to FineWeb-Edu and Lean, which use the GPT-2 vocabulary. DeepMind Mathematics uses the benchmark’s 73-symbol character vocabulary. It uses the same nonembedding parameter target but a smaller input-embedding/output matrix and hence fewer total parameters. We use depth-scaled initialization for projection matrices whose outputs are added to the residual stream, with the scale based on four block applications; dropout is zero. The TANGO implementation reduces memory use by processing destination queries in chunks rather than materializing every source–destination pair simultaneously. Transformer self-attention uses fused scaled-dot-product attention [6]. Query chunking and fused attention affect the implementation but not the model equations.

3 Experimental Design

Models, block sharing, and parameter matching. We compare six model architectures with approximately 44.3M nonembedding parameters. Each architecture performs four sequential block applications. The TANGO model, the WANGO model, and Recurrent Transformer++ reuse the same block parameters in all four applications; Untied Transformer++, full-attention GAU, and FLASH contain four independently parameterized blocks. Every model has $d_{\text{model}} = 512$ and, where applicable, 16 heads of width 32. We target 44,268,416 distinct nonembedding parameters, counting reused weights once. Because expansion widths must be integers, the resulting nonembedding parameter counts differ from this target by less than 0.1%. With the tied 50,257-entry GPT-2-vocabulary input/output matrix, this gives approximately 70M total parameters. DeepMind Mathematics instead uses the benchmark’s 73-symbol character vocabulary, so its nonembedding parameter target is unchanged but its embedding/output matrix is smaller.

We train each architecture three times. The (initialization, data-order) seed pairs are (17, 101), (23, 103), and (41, 107). For each pair, all architectures use the listed initialization seed and see exactly the same

examples in the same order under the listed data-order seed. We report the arithmetic mean and sample standard deviation of the validation NLL at the final checkpoint of each run. Every architecture receives the full training budget, with no architecture-specific checkpoint selection or early stopping. All likelihood results use teacher forcing: each prediction is conditioned on the ground-truth preceding tokens.

Eight-block comparison. We repeat all six architectures and three seed pairs with eight sequential block applications on FineWeb-Edu and Lean. Weight-shared architectures reuse one block eight times, whereas the other architectures use eight independent layers. The eight-block and four-block experiments use the same data splits, context lengths, training examples in the same order, optimization hyperparameters, and final-checkpoint evaluation procedure. This experiment contains 36 runs: six models, two datasets, and three seeds. We report validation NLL, including separate source-code and proof-completion losses for Lean. The weight-shared models retain the widths in table 1. Among the independently parameterized models, Untied Transformer++ uses $F = 2,912$, and FLASH uses $E = 3,552$. GAU uses $E = 3,536$ on FineWeb-Edu and $E = 3,552$ on Lean. The resulting nonembedding parameter counts range from 44,179,968 to 44,278,776. We do not report task-level generation metrics or empirical speed and memory measurements for these runs.

FineWeb-Edu. The natural-language experiment uses temporally partitioned FineWeb-Edu [18] and the GPT-2 tokenizer. Training documents come from Common Crawl dumps through CC-MAIN-2023-40; validation documents come from CC-MAIN-2023-50. To prevent duplicated documents from crossing the split, documents with the same normalized URL or identical UTF-8 content are assigned to the temporal partition containing their earliest occurrence.

Training documents are concatenated into fixed 8,192-token sequences. GPT-2’s end-of-text (EOT) token marks document boundaries but is masked as a prediction target. Validation consists of 2,048 sequences drawn wholly from individual documents, so no scored prediction crosses a document boundary. Each model trains for 5,722 optimizer steps with an effective batch size of 32 sequences, corresponding to 1.50 billion token positions processed. We report validation NLL per GPT-2 token.

Lean. The Lean experiment combines causal source-code modeling and proof completion using Mathlib commit `cf8e23a62939` [7, 27]. Both tasks use the same fixed file-level train/validation split, preventing declarations from one file from appearing in multiple splits. Source-code examples are 2,048-token segments contained within one file. For proof completion, the input contains up to the final 1,536 tokens preceding the proof. Loss is applied only to the complete proof, which may contain at most 511 GPT-2 tokens, and a terminal EOT token; prompt tokens are excluded from the loss.

Training alternates batches containing only source-code examples with batches containing only proof examples, giving the two tasks equal optimization weight. Each optimizer step has an effective batch size of 128 sequences. Training lasts 3,814 steps, corresponding to 999.817 million token positions processed, with half of the steps assigned to each task. Within each task, validation NLL averages over all supervised tokens. The reported joint Lean NLL is the arithmetic mean of the source-code and proof-completion NLLs, so the two tasks contribute equally.

DeepMind Mathematics. We use all 56 modules of the official DeepMind Mathematics suite [21] at easy, medium, and hard difficulty. We sample equally from the resulting 168 module–difficulty combinations. The question is provided as context, and loss is applied only to answer characters and the terminal end-of-sequence (EOS) symbol. All models use context length 256. Training lasts 84,000 optimizer steps with an effective batch size of 64 examples, corresponding to 5.376 million examples or 32,000 training examples per module–difficulty combination. The fixed validation set contains the same number of examples from every combination. We report validation NLL averaged over the supervised answer positions: the answer characters and the terminal EOS symbol.

Optimization. All runs use bfloat16 (BF16) model computation with 32-bit floating-point (FP32) optimizer state. We use AdamW with $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, peak learning rate 6×10^{-4} , cosine decay to 6×10^{-5} , weight decay 0.1, and global gradient-norm clipping at 1.0. Warmup lasts 1,680 optimizer steps for DeepMind Mathematics, 200 for FineWeb-Edu, and 76 for Lean. Weight decay is applied to learned matrices but not

Table 2: Validation negative log-likelihood (NLL) at the final training checkpoint. Entries are the mean $\pm$ sample standard deviation over three runs. Architectures with the same data-order seed receive the same examples in the same order. FineWeb-Edu is measured per GPT-2 token, Lean averages source-code and proof-completion NLL equally, and DeepMind Mathematics averages over answer characters and the terminal EOS symbol. Lower is better.

Model	FineWeb-Edu (8,192-token context; 5,722 steps)	Lean source + proof (2,048-token context; 3,814 steps)	DeepMind Mathematics (256-character context; 84,000 steps)
TANGO	3.293 ± 0.005	1.719 ± 0.026	0.473 ± 0.008
WANGO	3.355 ± 0.005	2.271 ± 0.017	0.487 ± 0.006
Recurrent Transformer++	3.392 ± 0.003	2.530 ± 0.041	0.537 ± 0.003
Untied Transformer++	3.318 ± 0.006	2.138 ± 0.033	0.556 ± 0.005
Full-attention GAU	3.484 ± 0.003	2.337 ± 0.043	0.515 ± 0.037
FLASH	3.465 ± 0.020	2.377 ± 0.030	0.557 ± 0.005

Table 3: Validation NLL for models with four sequential block applications. Entries are the mean $\pm$ sample standard deviation over three runs. For each data-order seed, every architecture receives the same examples in the same order. Joint NLL is the arithmetic mean of the source-code and proof-completion NLLs; lower is better.

Model	Joint NLL	Source-code NLL	Proof-completion NLL
TANGO	1.719 ± 0.026	1.883 ± 0.035	1.556 ± 0.018
WANGO	2.271 ± 0.017	2.544 ± 0.017	1.998 ± 0.023
Recurrent Transformer++	2.530 ± 0.041	2.875 ± 0.053	2.185 ± 0.029
Untied Transformer++	2.138 ± 0.033	2.387 ± 0.043	1.889 ± 0.025
Full-attention GAU	2.337 ± 0.043	2.624 ± 0.051	2.050 ± 0.036
FLASH	2.377 ± 0.030	2.697 ± 0.040	2.058 ± 0.024

to biases, normalization parameters, temperatures, or null-gate biases. Dropout is zero. Architectures may use different microbatch sizes to fit in GPU memory. Gradient accumulation keeps the effective number of sequences per optimizer step fixed. We weight the loss from each microbatch by its number of supervised targets so that supervised tokens retain the same relative weighting across microbatches.

Parameter and operation counts. For each architecture, we report the number of nonembedding parameters and the number of multiply-accumulate operations (MACs) calculated from the model equations for one forward pass. Reused parameters are counted once. The MAC count includes the output vocabulary projection, learned linear projections, computations over token pairs, and feed-forward layers. It excludes embedding lookup, normalization, masking, feature-map construction, and elementwise operations. These counts describe arithmetic work rather than measured runtime or GPU memory.

This paper does not report tokenizer-independent bits per byte, compiler-verified Lean proof completion, generated-answer exact match, or empirical training speed and memory. The quality comparisons therefore use only the validation NLL metrics defined above. In particular, likelihood on Lean proofs or mathematical answers should not be interpreted as a measurement of generated-proof correctness or exact-answer accuracy.

4 Results

Table 2 reports validation NLL from the checkpoint saved at the end of each benchmark’s training budget. TANGO has the lowest NLL on all three benchmarks. Among the architectures with linear sequence-length scaling, WANGO has the lowest FineWeb-Edu NLL. It also has lower FineWeb-Edu NLL than Recurrent Transformer++ at a similar analytical forward-pass operation count.

Table 3 reports three Lean validation NLLs under teacher forcing. Source-code NLL is evaluated on Mathlib source sequences, proof-completion NLL is evaluated on held-out proof targets, and joint NLL is their unweighted arithmetic mean. None of these measurements determines whether a generated proof is accepted by Lean.

Table 4: Validation NLL with eight sequential block applications. Entries are the mean $\pm$ sample standard deviation over three runs. For each data-order seed, every architecture receives the same examples in the same order; lower is better. Lean joint NLL gives equal weight to source-code modeling and proof completion.

Model	FineWeb-Edu NLL	Lean joint NLL	Lean source-code NLL	Lean proof-completion NLL
TANGO	3.212 $\pm$ 0.016	1.563 $\pm$ 0.024	1.695 $\pm$ 0.025	1.432 $\pm$ 0.023
WANGO	3.282 $\pm$ 0.001	2.231 $\pm$ 0.021	2.515 $\pm$ 0.026	1.948 $\pm$ 0.016
Recurrent Transformer++	3.325 $\pm$ 0.007	2.364 $\pm$ 0.013	2.678 $\pm$ 0.026	2.051 $\pm$ 0.014
Untied Transformer++	3.277 $\pm$ 0.010	1.803 $\pm$ 0.073	1.990 $\pm$ 0.082	1.616 $\pm$ 0.063
Full-attention GAU	3.323 $\pm$ 0.018	2.412 $\pm$ 0.062	2.717 $\pm$ 0.078	2.107 $\pm$ 0.047
FLASH	3.342 $\pm$ 0.006	2.314 $\pm$ 0.031	2.603 $\pm$ 0.038	2.024 $\pm$ 0.025

Table 5: Parameter counts and analytical forward-pass computation for models with four sequential block applications on one 8,192-token FineWeb-Edu sequence. Nonembedding parameters count reused weights once and exclude the tied input-embedding/output matrix. One MAC is one multiply-accumulate operation.

Model	Nonembedding params	MACs/sequence	Scaling with sequence length
TANGO	44.271M	9.44T	quadratic
WANGO	44.271M	1.90T	linear
Recurrent Transformer++	44.279M	1.94T	quadratic
Untied Transformer++	44.307M	848.60B	quadratic
Full-attention GAU	44.238M	2.52T	quadratic
FLASH	44.275M	693.98B	linear

4.1 Eight-block comparison

Table 4 reports the comparison with eight passes through a shared block or eight independent layers. It includes all 36 runs: six models, two datasets, and three seeds.

With eight block applications, TANGO has the lowest NLL on both FineWeb-Edu and Lean. On FineWeb-Edu, WANGO still has lower NLL than Recurrent Transformer++ and FLASH.

4.2 FineWeb-Edu

On FineWeb-Edu, TANGO has the lowest validation NLL overall at 3.293 ± 0.005 . Among the two architectures with computation linear in sequence length, WANGO has an NLL of 3.355 ± 0.005 , compared with 3.465 ± 0.020 for FLASH. Its NLL is also lower than 3.392 ± 0.003 for Recurrent Transformer++. The Untied Transformer++ obtains 3.318 ± 0.006 , and full-attention GAU obtains 3.484 ± 0.003 .

Table 5 reports the analytical operation count for one 8,192-token sequence. WANGO requires 1.90T MACs, compared with 1.94T MACs for Recurrent Transformer++ at this context length. The analytical operation counts are similar in this configuration, while the NLL is lower for WANGO. For longer sequences, its operation count grows linearly rather than quadratically when the window and feature dimensions are fixed. The table does not report wall-clock speed or GPU memory.

4.3 Lean and DeepMind Mathematics

On Lean, TANGO has the lowest joint validation NLL at 1.719 ± 0.026 . The next-lowest result is 2.138 ± 0.033 from the Untied Transformer++. Table 2 reports the corresponding joint NLL for every other architecture. Table 3 shows that TANGO also has the lowest NLL for source-code modeling and proof completion considered separately. The joint score is the unweighted arithmetic mean of these two NLLs. These measurements use teacher forcing and do not determine whether generated proofs are accepted by Lean.

TANGO also has the lowest DeepMind Mathematics validation NLL, at 0.473 ± 0.008 per supervised answer position (answer characters and the terminal EOS symbol), compared with 0.487 ± 0.006 for WANGO, 0.537 ± 0.003 for the Recurrent Transformer++, and 0.556 ± 0.005 for the Untied Transformer++. Unlike the Lean experiment, DeepMind Mathematics uses a 73-symbol character vocabulary, 256-character contexts, answer-only supervision, and equal sampling from 168 module-difficulty combinations.

5 Related Work

Cross-token gated computation. Transformers combine self-attention, which mixes information across positions, with a feed-forward network applied separately at each position [28]. The representation entering the feed-forward network already contains information from other positions, but its gate is computed from the representation at the destination. In our models, each source token produces a SwiGLU gate vector, and each destination applies a weighted average of the visible source gates to its own projected features. The models therefore mix information across positions within a single gated residual update that replaces both standard sublayers.

Parameter sharing across depth. Reusing a block at multiple depths has been studied in Universal Transformers, ALBERT, and deep-equilibrium models [2, 8, 17]. Recent looped language models also study repeated computation with shared weights [20], including variants with repetition conditioning, stochastic recurrence, or additional evaluations at test time [9, 30]. Other work compares models with parameter-shared blocks, independently parameterized blocks, or depth that is increased during training [15]. Our Recurrent Transformer++ baseline applies one modern decoder block four times with shared parameters. Untied Transformer++ instead uses four independently parameterized blocks. Both baselines use pre-RMSNorm, RoPE self-attention, and SwiGLU feed-forward networks.

Sequence models with linear cost in context length. Sparse attention restricts which positions interact [4, 32]. Kernelized and linear attention use feature maps that allow key-value statistics to be accumulated in running sums [5, 16], while recurrent state-space models summarize the prefix in a fixed-size state [10, 11]. We include GAU and FLASH as gated-attention baselines with different sequence-length complexity. GAU uses full pairwise attention, whereas FLASH combines full interaction within local chunks with a global linear-attention summary [14]. Gated Linear Attention uses input-dependent gates to control updates to its recurrent linear-attention state [31], and BASED combines linear attention with sliding-window attention [1].

WANGO also combines local pairwise interactions with a summary of the older prefix. For recent source positions, it uses the same unnormalized exponential cosine-similarity scores as TANGO. For older positions, it applies a positive feature map to each key and maintains running sums of the resulting key features and their outer products with source gates. Its computation is linear in sequence length for fixed window and feature dimensions. Unlike self-attention and linear attention, which use their weights to aggregate value vectors, WANGO uses them to aggregate SwiGLU gates computed at source positions.

Conditional computation. Hypernetworks generate parameters for another network [12]; Lambda layers turn contextual information into linear functions applied at individual positions [3]; and fast-weight systems store temporary transformations in a changing state [22, 23]. Mixture-of-experts models instead choose among a discrete or sparse set of learned modules [25]. Our model does not generate a general weight matrix. Each source representation produces a SwiGLU gate, and each destination uses a weighted average of the visible source gates to modulate its own projected features. These gates are activations computed for the current sequence; they do not alter the learned model parameters or select from a fixed set of experts.

Formal proofs and generated mathematics. DeepMind Mathematics evaluates generated symbolic answers across a broad family of mathematical tasks [21]. Lean is an interactive theorem prover, and Mathlib is a community-developed library of formalized mathematics for Lean. Lean can check a generated proof against a fixed version of the proof assistant, Mathlib, and its other dependencies. Prior work trained language models to generate proofs in Metamath [19]. In Lean, PACT trains language models on proof artifacts, while recent whole-proof systems use proof-assistant feedback and machine checking [13, 29]. These task-level evaluations check whether generated outputs are accepted, whereas held-out next-token NLL measures their average predictive likelihood under teacher forcing. We report NLL separately for Mathlib source code, Lean proof-completion targets, and DeepMind Mathematics answers. We do not submit generated proofs to Lean or measure exact match on generated mathematical answers.

6 Discussion

6.1 Validation NLL and computational cost

At the evaluated parameter scale, TANGO has the lowest validation NLL on FineWeb-Edu, Lean, and DeepMind Mathematics. In each benchmark with four sequential block applications, it also has the highest analytical operation count, although the size of the difference depends on context length. For an 8,192-token FineWeb-Edu sequence, its forward pass requires 9.44T multiply-accumulate operations (MACs), compared with 1.90T for WANGO and 1.94T for Recurrent Transformer++. At the 256-character context used for DeepMind Mathematics, the corresponding counts for TANGO and WANGO are 52.94B and 52.91B. The T^2F term in TANGO’s analytical operation count comes from aggregating an F -dimensional gate for every causal source-destination pair. In these experiments, $F = 28,480$.

On FineWeb-Edu, WANGO has lower validation NLL than Recurrent Transformer++ at a similar analytical operation count. It also has lower NLL than FLASH, the other architecture in the comparison whose computation is linear in sequence length. In the eight-block experiment, TANGO again has the lowest FineWeb-Edu NLL, and WANGO has lower NLL than Recurrent Transformer++ and FLASH. WANGO computes pairwise interactions within a fixed local window and represents older source gates with feature-map prefix sums. Its analytical operation count grows linearly with sequence length when the window and feature dimensions are fixed. At autoregressive inference, the running prefix statistics do not grow with sequence length under the same assumptions; a parallel training implementation may still store intermediate activations for every position.

These operation counts are analytical rather than empirical. We did not measure wall-clock training speed or GPU memory, and the reported counts do not include normalization, masking, feature-map construction, or elementwise operations. They should therefore be used to compare the arithmetic specified by the model equations, not implementation efficiency.

6.2 Relation to self-attention and the comparison architectures

Standard self-attention uses source-dependent weights to average value vectors and then applies a feed-forward network separately at each destination. In TANGO and WANGO, source-dependent weights instead average SwiGLU gates produced at source positions. The averaged gate is multiplied elementwise by the destination’s projected features. A source token can therefore change the feature-wise scaling applied at a destination, but it does not generate a full weight matrix or an independent neural network. Both models use the same gated update and differ only in how they aggregate recent and older source gates.

Recurrent Transformer++ shares one block across four applications but uses conventional self-attention and SwiGLU applied independently at each position. Untied Transformer++ uses four separate Transformer blocks. GAU and FLASH apply gating within their attention computations, whereas TANGO and WANGO average SwiGLU gates computed at source positions. These comparisons vary several architectural components at once, so they do not establish that cross-token gate aggregation alone caused the observed NLL differences.

6.3 Limitations

The four-block experiments cover one parameter scale. The eight-block experiments include only FineWeb-Edu and Lean. Nonembedding parameter counts are within 0.1% of a common target. We also match training objectives, examples and their order, optimization hyperparameters, context lengths, and the number of sequential block applications. We do not match forward-pass operation counts, and these experiments are not sufficient to establish scaling laws. At the longer FineWeb-Edu and Lean contexts, TANGO performs substantially more arithmetic than WANGO; their operation counts are nearly equal at the shorter DeepMind Mathematics context. The lower validation NLL of TANGO should not be interpreted as a speed or memory advantage or as a compute-matched improvement.

We also do not report tokenizer-independent bits per byte, compiler-verified Lean proof completion, generated-answer exact match, empirical training speed, or GPU memory. Validation likelihood is not a substitute for those measurements. Our conclusions concern only validation NLL and analytical operation counts for the tested model size, data splits, and training budgets.

7 Conclusion

We studied a decoder block that replaces the usual self-attention and position-wise feed-forward sublayers with one cross-token gated update. Each source token produces a SwiGLU gate, and each destination applies a weighted average of source gates to its own projected features. TANGO aggregates gates from every causally visible source and has quadratic sequence-length complexity. WANGO retains pairwise interactions within a fixed local window and uses feature-map prefix sums for older sources. Its sequence-length complexity is linear when the window and feature dimensions are fixed.

At the evaluated parameter scale, TANGO has the lowest validation NLL on FineWeb-Edu, Lean, and DeepMind Mathematics. On FineWeb-Edu, WANGO has the lowest NLL among the tested architectures with linear sequence-length complexity and has lower NLL than Recurrent Transformer++ at a similar analytical operation count. The experiments measure next-token likelihood and analytical operations; they do not measure generated-answer accuracy, proof acceptance, wall-clock speed, or GPU memory.

References

- [1] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalina, Silas Alberti, James Zou, Atri Rudra, and Christopher Ré. Simple linear attention language models balance the recall-throughput tradeoff. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 1763–1840, 2024. URL <https://proceedings.mlr.press/v235/arora24a.html>.
- [2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, 2019.
- [3] Irwan Bello. LambdaNetworks: Modeling long-range interactions without attention. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=xTJEN-gg11b>.
- [4] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [5] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations*, 2021.
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [7] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, 2021.
- [8] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.
- [9] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *Advances in Neural Information Processing Systems*, 2025. URL <https://arxiv.org/abs/2502.05171>.
- [10] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling*, 2024.
- [11] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

- [12] David Ha, Andrew M. Dai, and Quoc V. Le. Hypernetworks. In *International Conference on Learning Representations*, 2017.
- [13] Jesse Michael Han, Jason Rute, Yuhuai Wu, Edward W. Ayers, and Stanislas Polu. Proof artifact co-training for theorem proving with language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=rpxJc9j04U>.
- [14] Weizhe Hua, Zihang Dai, Hanxiao Liu, and Quoc V. Le. Transformer quality in linear time. In *International Conference on Machine Learning*, 2022.
- [15] Ferdinand Kapl, Emmanouil Angelis, Kaitlin Maile, Johannes von Oswald, and Stefan Bauer. From growing to looping: A unified view of iterative computation in LLMs. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2602.16490>.
- [16] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, 2020.
- [17] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations*, 2020.
- [18] Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- [19] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.
- [20] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=din0lGfZFd>.
- [21] David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. Analysing mathematical reasoning abilities of neural models. In *International Conference on Learning Representations*, 2019.
- [22] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, 2021.
- [23] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- [24] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [25] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.
- [26] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- [27] The mathlib Community. The Lean mathematical library. In *Certified Programs and Proofs*, 2020.
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

- [29] Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, Wenjun Gao, Haowei Zhang, Qihao Zhu, Dejian Yang, Zhibin Gou, Z. F. Wu, Fuli Luo, and Chong Ruan. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=I4YAIwrsXa>.
- [30] Kevin Xu and Issei Sato. On expressive power of looped transformers: Theoretical analysis and enhancement via timestep encoding. In *International Conference on Machine Learning*, 2025. URL <https://proceedings.mlr.press/v267/xu25x.html>.
- [31] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 56501–56523, 2024. URL <https://proceedings.mlr.press/v235/yang24ab.html>.
- [32] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.